The proofgraph package

Pierre Senellart
pierre@senellart.com

2026/05/31 v0.1.0

Abstract

The proofgraph package automatically produces a graph of the dependencies between the results (theorems, lemmas, propositions) of a mathematical article. It infers an edge from one result to another whenever the proof of the former refers to the latter through an ordinary cross-reference, requiring no manual annotation. The graph is written as a Graphviz `.dot` file, which can be rendered externally or, optionally, embedded into the document automatically.

1 Introduction

When writing a mathematical article, it is often useful to visualise how the results depend on one another: which lemma is used in the proof of which theorem, and so on. proofgraph builds such a dependency graph automatically. Each theorem-like environment becomes a node; each cross-reference (`\ref`, `\cref`, ...) appearing *inside a proof* becomes an edge from the proved result to the referred-to result.

2 Usage

Load the package after any theorem-providing package (such as amsthm) and *before* your `\newtheorem` declarations:

```
\usepackage{proofgraph}
\newtheorem{theorem}{Theorem}
\newtheorem{lemma}{Lemma}
```

Then write your document as usual. After two compilation runs, a file `\jobname.dot` is produced. Render it with Graphviz:

```
dot -Tpdf myfile.dot -o mygraph.pdf
```

`\uses` Inside a proof, `\uses{`⟨*label*⟩`}` records a dependency on the result labelled ⟨*label*⟩ even when no visible reference is made. A comma-separated list is accepted: `\uses{lem:a,lem:b}`.

`\proofgraphedge` `\proofgraphedge{`⟨A⟩`}{`⟨B⟩`}` adds an edge recording that result ⟨A⟩ depends on result ⟨B⟩, like a `\uses` but stated outside any proof. This is handy for an "obvious" corollary that relies on earlier results without a `proof` environment:

`\proofgraphedge{cor:x}{thm:y}`. It may appear anywhere; both labels are resolved when the graph is written.

`\proofof` By default a proof is attached to the most recently started result, and deferred proofs (as with apxproof) are attached automatically to the result they prove. When a proof does not directly follow its result, `\proofof{` ⟨*label*⟩`}` inside the proof pins it to the result labelled ⟨*label*⟩.

`\proofgraphtrack` `\proofgraphuntrack` Result environments are detected automatically. Expository environments are excluded from the graph by default (`definition`, `example`, `remark`, `note`). In the preamble, `\proofgraphuntrack{`⟨*names*⟩`}` excludes further environment types and `\proofgraphtrack{`⟨*names*⟩`}` forces environments to be tracked (overriding the default exclusions), e.g. `\proofgraphuntrack{observation}` or `\proofgraphtrack{definition}`. Nodes are labelled by the result's formatted name and number ("Theorem 1"), not by the environment name.

`\proofgraphignore` `\proofgraphignore{`⟨*from*⟩`}{`⟨*to*⟩`}` suppresses the edge from ⟨*from*⟩ to ⟨*to*⟩ (e.g. an unwanted forward citation).

`\proofgraphexclude` `\proofgraphexclude{`⟨*label*⟩`}` removes the result labelled ⟨*label*⟩, together with all its incident edges, from the graph.

`\proofgraphstyle` `\proofgraphstyle{`⟨*env*⟩`}{`⟨*attrs*⟩`}` sets Graphviz node attributes for all results of the environment ⟨*env*⟩, e.g. `\proofgraphstyle{lemma}{shape=ellipse,color=blue}`.

`\proofgraphstylecite` `\proofgraphstylecite{`⟨*attrs*⟩`}` sets the Graphviz attributes of the external citation nodes added by the `cite` option (default `shape=note`), e.g. `\proofgraphstylecite{shape=box,st`

`\proofgraph` In `autorun` mode, `\proofgraph` includes the rendered graph at that point (using the image produced on the previous run).

2.1 Options

option `selfloops` `selfloops=remove` (default) drops edges from a result to itself; `selfloops=keep` retains them.

option `autorun` `autorun` (boolean, default false) runs Graphviz automatically at the end of the run; requires compilation with shell-escape enabled.

option `engine` Graphviz layout engine (default `dot`).

option `format` Output format for `autorun`/`\proofgraph` (default `pdf`).

option `direction` `direction=usedby` (default) orients an edge from the result that is used to the result that uses it, so an arrow `a->b` reads "`a` is used by `b`" (`b` relies on `a`). `direction=uses` reverses it (`a->b` means "`a` uses `b`").

option `rankdir` Graph direction (default `LR`).

option `cite` `cite` (boolean, default false) also captures `\cite` inside proofs as dependencies on external work, drawn as distinct nodes. A `\cite` optional note is kept: `\cite[Thm~3]{key}` labels the node "Thm 3, . . . ".

option `citelabel` `citelabel=key` (default) labels citation nodes by their BibTeX key; `citelabel=tag` uses the printed citation tag instead ("[1]", "[Knu74]", "[Abiteboul et al. 1995]", . . .), recovered from the `.aux` (so it needs two runs). It supports the standard, hyperref-wrapped and natbib (numeric and author–year) bibliographies; if no clean tag can be recovered (e.g. before the first BibTeX run) it keeps the key.

option `file` Base name of the output file (default `\jobname`).

2.2 Use with apxproof

apxproof moves proofs to the appendix and typesets them there, so the references a proof makes are only executed at that later point. To attribute them to the right

result, proofgraph relies on the hook `\apxproofhook`, which apxproof fires inside each deferred proof. This hook is available in apxproof **v1.4.1** and later; load the two packages in either order and no further setup is needed.

With an *earlier* apxproof the document still compiles cleanly and every result still becomes a node, but *no proof dependencies are captured* (the graph has nodes but no proof edges), because proofgraph cannot tell which appendix proof belongs to which result. In that case, add the dependencies manually with `\uses` (and `\proofof` to pin a proof to its result), or upgrade apxproof.

3 Limitations

Nodes are created only for environments declared through `\newtheorem` *after* the package is loaded. Results must be labelled for references to them to be resolved. Two compilation runs are required, as for any cross-reference. Theorem titles containing the characters `"` or `\` in the node label are not escaped in this version.

4 Implementation

```
⟨*package⟩
\NeedsTeXFormat{LaTeX2e}[2005/12/01]
\ProvidesPackage{proofgraph}
  [2026/05/31 v0.1.0 Dependency graph of results]
```

4.1 Dependencies

```
\RequirePackage{amsthm}
\RequirePackage{etoolbox}
\RequirePackage{xstring}
\RequirePackage{kvoptions}
\RequirePackage{graphicx}
```

`\pgph@trimsp` An expandable space-trimmer (labels inside a `\cite`/`\cref`/`\uses` list arrive with the space after each comma), built on the `expl3` primitive so it is robust to leading spaces and braces.

```
\ExplSyntaxOn
\cs_new:Npn \pgph@trimsp #1 { \tl_trim_spaces:n {#1} }
\ExplSyntaxOff
```

4.2 Options

```
\SetupKeyvalOptions{family=pgph,prefix=pgph@}
\DeclareStringOption[remove]{selfloops}
\DeclareBoolOption{autorun}
\DeclareStringOption[dot]{engine}
\DeclareStringOption[pdf]{format}
\DeclareStringOption[LR]{rankdir}
\DeclareStringOption[usedby]{direction}
\DeclareBoolOption{cite}
\DeclareStringOption[key]{citelabel}
\DeclareStringOption{file}
\ProcessKeyvalOptions*
\ifx\pgph@file\@empty\def\pgph@file{\jobname}\fi
```

4.3 Literal braces

We need catcode-12 braces to write the Graphviz `digraph { }` block.

```
\begingroup
  \catcode`\[=1 \catcode`\]=2
  \catcode`\{=12 \catcode`\}=12
  \gdef\pgph@ob[{]%
  \gdef\pgph@cb[}]%
\endgroup
```

4.4 Data structures

A global counter gives each result instance a unique numeric id. Nodes and edges are accumulated in list macros and emitted at end of document, so that editing commands (ignore, exclude, self-loops) can be applied to the complete graph regardless of where they appear.

```
\newcount\pgph@idcnt
\def\pgph@nodes{}
\def\pgph@edges{}
\def\pgph@ignores{}
\def\pgph@excludes{}
\let\pgph@curresult\@empty
\let\pgph@curproof\@empty
```

`\pgph@auxmap` The label-to-id map. `\label` inside a result records, both in memory and in the `.aux` file, that the user label maps to the current node id, so references resolve independently of hyperref.

```
\newcommand\pgph@auxmap[2]{\global\@namedef{pgph@map@#1}{#2}}
\newcommand\pgph@setmap[2]{%
  \edef\pgph@tmpid{#2}%
  \global\expandafter\edef\csname pgph@map@#1\endcsname{\pgph@tmpid}%
  \protected@write\@auxout{}{\string\pgph@auxmap{#1}{\pgph@tmpid}}%
}
```

4.5 Registering result environments

`\pgph@register` A result environment is tracked by installing a begin-hook that records a node. In every declaration command (`\newtheorem`, `\spnewtheorem`) the environment *name* is the first mandatory argument, so registration does not need to parse the remaining (varied) arguments. Each name met is queued as a *candidate* (deduplicated); the begin-hooks themselves are installed only at `\begin{document}`, once the inclusion and exclusion lists are settled, so the customisation commands below may appear anywhere in the preamble.

```
\let\pgph@candidates\@empty
\newcommand\pgph@register[1]{%
  \edef\pgph@thisname{#1}%
  \IfBeginWith\pgph@thisname{axp@}{}{%
    \ifcsname pgph@cand@\pgph@thisname\endcsname\else
      \global\@namedef{pgph@cand@\pgph@thisname}{}%
      \ifx\pgph@candidates\@empty
        \global\let\pgph@candidates\pgph@thisname
      \else
```

```
        \xdef\pgph@candidates{\pgph@candidates,\pgph@thisname}%
      \fi
    \fi
  }%
}
```

We wrap `\newtheorem` (and, for Springer classes, `\spnewtheorem`) to register each newly declared environment, replaying the original command so its own argument parsing is untouched. The starred (unnumbered) forms are *not* auto-registered: a results graph wants numbered statements, and these forms are commonly used for unnumbered repeats of a result (e.g. the appendix copies produced by apxproof and myproofs), which would otherwise appear as spurious unnumbered duplicate nodes. Use `\proofgraphtrack` to track an unnumbered environment deliberately.

```
\let\pgph@orig@newtheorem\newtheorem
\renewcommand\newtheorem{\@ifstar{\pgph@nt@star}{\pgph@nt@plain}}
\newcommand\pgph@nt@plain[1]{\pgph@register{#1}\pgph@orig@newtheorem{#1}}
\newcommand\pgph@nt@star[1]{\pgph@orig@newtheorem*{#1}}
\ifdefined\spnewtheorem
  \let\pgph@orig@spnewtheorem\spnewtheorem
  \renewcommand\spnewtheorem{\@ifstar{\pgph@spnt@star}{\pgph@spnt@plain}}
  \newcommand\pgph@spnt@plain[1]{\pgph@register{#1}\pgph@orig@spnewtheorem{#1}}
  \newcommand\pgph@spnt@star[1]{\pgph@orig@spnewtheorem*{#1}}
\fi
```

`\proofgraphtrack`
`\proofgraphuntrack`

Result environments are detected automatically, but the analysis can be tuned. `\proofgraphtrack{`⟨*names*⟩`}` forces the listed environments to be tracked, overriding any exclusion; `\proofgraphuntrack{`⟨*names*⟩`}` excludes the listed environment *types*. Expository environments are excluded by default: `definition`, `example`, `remark`, `note`. Both commands belong in the preamble. `\pgph@defaultenvs` is the list of class-predefined names probed at `\begin{document}`; `\pgph@defaultexclude` seeds the exclusion set.

```
\newcommand\pgph@defaultenvs{theorem,lemma,proposition,corollary,definition,%
  conjecture,claim,fact,remark,example,observation,notation,problem,%
  question,property,assumption,hypothesis,principle}
\newcommand\pgph@defaultexclude{definition,example,remark,note}
\newcommand\proofgraphtrack[1]{%
  \@for\pgph@one:=#1\do{%
    \csundef{pgph@xtype@\pgph@one}%
    \global\@namedef{pgph@force@\pgph@one}{}%
    \expandafter\pgph@register\expandafter{\pgph@one}%
  }%
}
\newcommand\proofgraphuntrack[1]{%
  \@for\pgph@one:=#1\do{%
    \csundef{pgph@force@\pgph@one}%
    \global\@namedef{pgph@xtype@\pgph@one}{}%
  }%
}
\expandafter\proofgraphuntrack\expandafter{\pgph@defaultexclude}
```

Environments predefined by the document class (acmart, lipics, Springer

classes) are declared before the package is loaded, so the wrappers above never see them. At `\begin{document}` we add every common result name that exists to the candidate list, then install a begin-hook for each candidate that is forced or not excluded.

```
\newcommand\pgph@dohook[1]{%
  \ifcsname pgph@hooked@#1\endcsname\else
    \global\@namedef{pgph@hooked@#1}{}%
    \AtBeginEnvironment{#1}{\pgph@beginresult{#1}}%
  \fi
}
\newcommand\pgph@maybehook[1]{%
  \ifcsname pgph@force@#1\endcsname
    \pgph@dohook{#1}%
  \else
    \ifcsname pgph@xtype@#1\endcsname\else\pgph@dohook{#1}\fi
  \fi
}
\newcommand\pgph@resolveregistration[1]{%
  \@for\pgph@one:=#1\do{%
    \ifcsname\pgph@one\endcsname\expandafter\pgph@register\expandafter{\pgph@one}\fi}%
  \ifx\pgph@candidates\@empty\else
    \@for\pgph@one:=\pgph@candidates\do{%
      \expandafter\pgph@maybehook\expandafter{\pgph@one}}%
  \fi
}
\AtBeginDocument{%
  \expandafter\pgph@resolveregistration\expandafter{\pgph@defaultenvs}}
```

`\pgph@beginresult` `\pgph@hooknames` `\pgph@recordresult`

Start a result: allocate an id and record the node. The node is keyed and styled by the environment name (third argument, also a fallback label), but displayed by the result's *formatted* name ("Theorem", "Lemma"), captured from the heading. We get that name by locally wrapping `\@begintheorem`/`\@opargbegintheorem` (the hooks every `amsthm`- and `\newtheorem`-based class routes its heading through), whose first argument is the title. These redefinitions are confined to the current environment group, so they revert at `\end`. We also locally redefine `\label` to populate the map.

Nodes are accumulated in *reverse* order of appearance (`\pgph@prependnode`). With `rankdir=LR`, Graphviz stacks disconnected components vertically in the order they are read, from the bottom up; emitting the results last-to-first therefore lays them out top-to-bottom in document order (first result on top), while connected results stay grouped. This is a layout tendency, not a guarantee, but needs no `pack` trickery.

```
\newcommand\pgph@prependnode[1]{%
  \xdef\pgph@nodes{\unexpanded{#1}\unexpanded\expandafter{\pgph@nodes}}%
}
\newcommand\pgph@beginresult[1]{%
  \global\advance\pgph@idcnt\@ne
  \xdef\pgph@curresult{\the\pgph@idcnt}%
  \protected@edef\pgph@tmp{%
    \noexpand\pgph@node{\pgph@curresult}{#1}{#1}}%
  \expandafter\pgph@prependnode\expandafter{\pgph@tmp}%
  \pgph@hooknames
```

```
  \ifx\label\pgph@maplabel\else\let\pgph@savedlabel\label\fi
  \let\label\pgph@maplabel
}
\newcommand\pgph@hooknames{%
  \ifdefined\@begintheorem
    \let\pgph@orig@bt\@begintheorem
    \def\@begintheorem##1##2{\pgph@recordresult{##1}{##2}\pgph@orig@bt{##1}{##2}}%
  \fi
  \ifdefined\@opargbegintheorem
    \let\pgph@orig@obt\@opargbegintheorem
    \def\@opargbegintheorem##1##2##3{%
      \pgph@recordresult{##1}{##2}\pgph@orig@obt{##1}{##2}{##3}}%
  \fi
  \ifdefined\@spbegintheorem
    \let\pgph@orig@spbt\@spbegintheorem
    \def\@spbegintheorem##1##2##3##4{%
      \pgph@recordresult{##1}{##2}\pgph@orig@spbt{##1}{##2}{##3}{##4}}%
  \fi
  \ifdefined\@spopargbegintheorem
    \let\pgph@orig@spobt\@spopargbegintheorem
    \def\@spopargbegintheorem##1##2##3##4##5{%
      \pgph@recordresult{##1}{##2}\pgph@orig@spobt{##1}{##2}{##3}{##4}{##5}}%
  \fi
}
```

The formatted name is the first argument of every heading macro (`\@begintheorem`/`\@opargbegintheore` for `amsthm`/`\newtheorem`, `\@spbegintheorem`/`\@spopargbegintheorem` for Springer's `\spnewtheorem`); the second argument is the number. We use that second argument only to tell *whether* the result is numbered (it is empty for the starred, unnumbered forms): an unnumbered result records no number, so it is dropped at emission unless tracked explicitly. The number *value*, when present, is read from `\@currentlabel` rather than from that argument, which some classes pollute (e.g. lipics injects `\advance\par@deathcycles\@ne`). `\@currentlabel`, set by the result's `\refstepcounter` just before the heading, is the clean number and needs no `\label`. Values are reduced to plain strings now, with fragile wrappers neutralized (notably apxproof's forward-link). The first heading wins (guarded on the name), ignoring nested headings. The emptiness of the number argument is tested without expanding it, so a polluted (but non-empty) number does not trip the test.

```
\newcommand\pgph@recordresult[2]{%
  \ifcsname pgph@name@\pgph@curresult\endcsname\else
    \begingroup
      \let\protect\@empty
      \def\axp@forward@link##1##2{##2}%
      \edef\pgph@tmpname{#1}%
      \global\expandafter\let\csname pgph@name@\pgph@curresult\endcsname
                             \pgph@tmpname
      \global\@namedef{pgph@seen@\pgph@curresult}{}%
      \def\pgph@tmphn{#2}%
      \ifx\pgph@tmphn\@empty\else
        \edef\pgph@tmpnum{\@currentlabel}%
        \global\expandafter\let\csname pgph@num@\pgph@curresult\endcsname
                               \pgph@tmpnum
```

```
      \fi
    \endgroup
  \fi
}
```

`\pgph@maplabel` records the label-to-node mapping. As a fallback for classes whose headings pass through none of the hooked heading macros (so no heading was `seen`), it also captures the number from `\@currentlabel`. We do *not* do this once a heading was seen: there, an absent number means the result is genuinely unnumbered (a starred form), and `\@currentlabel` could be a stale value from an earlier result. The value is expanded to a plain string *now*, inside a group where fragile wrappers are neutralized, so that nothing dangerous survives to the emission pass: in particular, under apxproof forward-linking, `\@currentlabel` holds `\axp@forward@link{`⟨*target*⟩`}{`⟨*number*⟩`}`, whose hyperlink machinery must never be expanded in a non-typesetting context (it would run away). Reducing it to its number argument keeps it harmless.

`\pgph@maplabel`

```
\newcommand\pgph@maplabel[1]{%
  \pgph@setmap{#1}{\pgph@curresult}%
  \ifcsname pgph@num@\pgph@curresult\endcsname\else
   \ifcsname pgph@seen@\pgph@curresult\endcsname\else
    \begingroup
      \let\protect\@empty
      \def\axp@forward@link##1##2{##2}%
      \edef\pgph@tmpnum{\@currentlabel}%
      \global\expandafter\let\csname pgph@num@\pgph@curresult\endcsname\pgph@tmpnum
    \endgroup
   \fi
  \fi
  \pgph@savedlabel{#1}%
}
```

4.6 Hooking proofs

Inside a proof we redefine the reference commands to capture edges from the proved result.

```
\newcommand\pgph@addedge[2]{%
  \protected@edef\pgph@tmpedge{\noexpand\pgph@edge{#1}{#2}}%
  \expandafter\g@addto@macro\expandafter\pgph@edges\expandafter{\pgph@tmpedge}%
}
\newcommand\pgph@recordref[1]{%
  \ifx\pgph@curproof\@empty\else
    \@for\pgph@one:=#1\do{%
      \edef\pgph@k{\expandafter\pgph@trimsp\expandafter{\pgph@one}}%
      \pgph@addedge{\pgph@curproof}{\pgph@k}}%
  \fi
}
```

`\pgph@beginproof` Attach the proof to the current result and install the capturing versions of the reference commands (only those that exist).

The capturing commands are `\protected` so that, when a reference appears in a moving argument inside a proof (e.g. a `\caption`), they do not expand into the `.aux`/`.lof` and leak internal tokens.

```
\protected\def\pgph@cap@ref#1{\pgph@recordref{#1}\pgph@saved@ref{#1}}
\protected\def\pgph@cap@cref#1{\pgph@recordref{#1}\pgph@saved@cref{#1}}
\protected\def\pgph@cap@Cref#1{\pgph@recordref{#1}\pgph@saved@Cref{#1}}
\protected\def\pgph@cap@autoref#1{\pgph@recordref{#1}\pgph@saved@autoref{#1}}
\protected\def\pgph@cap@eqref#1{\pgph@recordref{#1}\pgph@saved@eqref{#1}}
\protected\def\pgph@cap@vref#1{\pgph@recordref{#1}\pgph@saved@vref{#1}}
\newcommand\pgph@install[1]{%
  \ifcsname #1\endcsname
    \expandafter\ifx\csname #1\expandafter\endcsname\csname pgph@cap@#1\endcsname
    \else
      \expandafter\let\csname pgph@saved@#1\expandafter\endcsname\csname #1\endcsname
      \expandafter\let\csname #1\expandafter\endcsname\csname pgph@cap@#1\endcsname
    \fi
  \fi
}
\newcommand\pgph@beginproof{%
  \let\pgph@curproof\pgph@curresult
  \pgph@install{ref}\pgph@install{cref}\pgph@install{Cref}%
  \pgph@install{autoref}\pgph@install{eqref}\pgph@install{vref}%
  \ifpgph@cite\pgph@installcite\fi
}
```

Packages such as apxproof capture the body of a `proof` verbatim (to defer it to the appendix) and replay it there. Patching the `proof` environment then corrupts that capture (its `\proof` has delimited arguments), and the references are only executed at replay time anyway. So the standard begin-`proof` hook is installed only when no such package is active; otherwise the deferred-proof capture (below) takes over. The registration happens at `\begin{document}`, once we know what is loaded.

```
\newif\ifpgph@apxproof
\AtBeginDocument{%
  \@ifpackageloaded{apxproof}%
    {\pgph@apxprooftrue\pgph@apxinit}%
    {\AtBeginEnvironment{proof}{\pgph@beginproof}}%
}
```

`\pgph@apxinit` `\pgph@apxproofcapture`

Under apxproof, deferred proofs are typeset in the appendix, where their references finally execute. apxproof fires `\apxproofhook` inside each such proof, passing the `axp@r`⟨*n*⟩ label of the proved theorem, which (because apxproof also attaches that label to the theorem in the main text) is already in our map. We resolve it to the source node and install the reference-capturing commands for the duration of the proof.

```
\newcommand\pgph@apxinit{\def\apxproofhook##1{\pgph@apxproofcapture{##1}}}
\newcommand\pgph@apxproofcapture[1]{%
  \edef\pgph@curproof{\pgph@resolve{#1}}%
  \ifx\pgph@curproof\@empty\else
    \pgph@install{ref}\pgph@install{cref}\pgph@install{Cref}%
    \pgph@install{autoref}\pgph@install{eqref}\pgph@install{vref}%
    \ifpgph@cite\pgph@installcite\fi
  \fi
```

```
}
```

`\pgph@hookcite` Optional `\cite` capture: external references become `cite:`-prefixed target labels, drawn later as distinct nodes. A `\cite` optional note, `\cite[⟨note⟩]{⟨key⟩}`, is recorded for the key(s) and shown in the node label as "⟨*note*⟩, ⟨*key*⟩". The note is reduced to a plain string at capture time (fragile wrappers neutralized); the first note seen for a key wins.

```
\newcommand\pgph@cap@cite[2][]{%
  \ifx\pgph@curproof\@empty\else
    \def\pgph@citenotearg{#1}%
    \@for\pgph@one:=#2\do{%
      \edef\pgph@k{\expandafter\pgph@trimsp\expandafter{\pgph@one}}%
      \pgph@addedge{\pgph@curproof}{cite:\pgph@k}%
      \ifx\pgph@citenotearg\@empty\else
        \expandafter\pgph@recordcitenote\expandafter{\pgph@k}{#1}%
      \fi}%
  \fi
  \pgph@saved@cite[#1]{#2}%
}
\newcommand\pgph@recordcitenote[2]{%
  \ifcsname pgph@citenote@#1\endcsname\else
    \begingroup
      \pgph@citesanitize
      \edef\pgph@tmpnote{#2}%
      \global\expandafter\let\csname pgph@citenote@#1\endcsname\pgph@tmpnote
    \endgroup
  \fi
}
\newcommand\pgph@installcite{%
  \ifcsname cite\endcsname
    \let\pgph@saved@cite\cite
    \let\cite\pgph@cap@cite
  \fi
}
```

4.7 User commands

```
\newcommand\uses[1]{\pgph@recordref{#1}}
\newcommand\proofof[1]{%
  \ifcsname pgph@map@#1\endcsname
    \xdef\pgph@curproof{\csname pgph@map@#1\endcsname}%
  \fi
}
\newcommand\proofgraphedge[2]{%
  \g@addto@macro\pgph@edges{\pgph@labeledge{#1}{#2}}%
}
\newcommand\pgph@labeledge[2]{%
  \edef\pgph@ef{\pgph@resolve{#1}}%
  \ifx\pgph@ef\@empty\else
    \expandafter\pgph@plainedge\expandafter{\pgph@ef}{#2}%
  \fi
}
\newcommand\proofgraphignore[2]{%
  \g@addto@macro\pgph@ignores{\pgph@ignorerule{#1}{#2}}%
```

```
}
\newcommand\proofgraphexclude[1]{%
  \g@addto@macro\pgph@excludes{\pgph@excluderule{#1}}%
}
\newcommand\proofgraphstyle[2]{\global\@namedef{pgph@style@#1}{#2}}
```

External citation nodes (option `cite`) share one style, held in `\pgph@citestyle` and overridable with `\proofgraphstylecite`; it defaults to `shape=note`.

```
\newcommand\pgph@citestyle{shape=note}
\newcommand\proofgraphstylecite[1]{\renewcommand\pgph@citestyle{#1}}
```

4.8 Resolution helpers

These run at end of document, after the `.aux` has populated the map.

```
\newcommand\pgph@resolve[1]{%
  \ifcsname pgph@map@#1\endcsname\csname pgph@map@#1\endcsname\fi
}
\newcommand\pgph@ignorerule[2]{%
  \edef\pgph@f{\pgph@resolve{#1}}\edef\pgph@t{\pgph@resolve{#2}}%
  \ifx\pgph@f\@empty\else\ifx\pgph@t\@empty\else
    \@namedef{pgph@ig@\pgph@f @\pgph@t}{}%
  \fi\fi
}
\newcommand\pgph@excluderule[1]{%
  \edef\pgph@x{\pgph@resolve{#1}}%
  \ifx\pgph@x\@empty\else\@namedef{pgph@ex@\pgph@x}{}\fi
}
```

4.9 Emission

`\pgph@node` Emit one node line, honouring exclusions and per-environment styling. External (`cite:`) targets are emitted separately as a pass over edges. An *unnumbered* result (no number captured) is omitted: a results graph is about numbered statements, and unnumbered theorem-like environments are typically expository or repeated copies (e.g. a class-predefined `\spnewtheorem*{claim}`). `\proofgraphtrack{`⟨*env*⟩`}` overrides this, so a deliberately unnumbered environment is still drawn.

```
\newcommand\pgph@node[3]{%
  \ifcsname pgph@ex@#1\endcsname\else
    \edef\pgph@nm{\ifcsname pgph@num@#1\endcsname\csname pgph@num@#1\endcsname\fi}%
    \ifx\pgph@nm\@empty
      \ifcsname pgph@force@#2\endcsname
        \pgph@emitnode{#1}{#2}{#3}%
      \else
        \@namedef{pgph@ex@#1}{}%
      \fi
    \else
      \pgph@emitnode{#1}{#2}{#3}%
    \fi
  \fi
}
\newcommand\pgph@emitnode[3]{%
  \edef\pgph@s{\ifcsname pgph@style@#2\endcsname
    , \csname pgph@style@#2\endcsname\fi}%
```

```
  \edef\pgph@dn{\ifcsname pgph@name@#1\endcsname
    \csname pgph@name@#1\endcsname\else#3\fi}%
  \immediate\write\pgph@out{%
    \space\space"n#1" [label="\pgph@dn\space\pgph@nm"\pgph@s];}%
}
```

`\pgph@edge` Emit one edge, resolving the target label to an id and applying the self-loop, ignore and exclude filters. Targets beginning with `cite:` denote external nodes.

```
\newcommand\pgph@edge[2]{%
  \IfBeginWith{#2}{cite:}%
    {\pgph@citeedge{#1}{#2}}%
    {\pgph@plainedge{#1}{#2}}%
}
\newcommand\pgph@plainedge[2]{%
  \edef\pgph@t{\pgph@resolve{#2}}%
  \ifx\pgph@t\@empty\else
    \ifcsname pgph@ex@#1\endcsname\else
    \ifcsname pgph@ex@\pgph@t\endcsname\else
    \ifcsname pgph@ig@#1@\pgph@t\endcsname\else
      \ifboolexpr{ test {\ifnumequal{#1}{\pgph@t}}
                   and test {\ifdefstring{\pgph@selfloops}{remove}} }%
        {}{\pgph@emitedge{n#1}{n\pgph@t}}%
    \fi\fi\fi
  \fi
}
```

`\pgph@emitedge` `\pgph@writeedge` The relation recorded is always "the result whose proof we are in (the first argument) uses the target (the second)". The `direction` option chooses the arrow orientation: `direction=usedby` (default) draws *target to result*, so an arrow `a->b` reads "`a` is used by `b`" (i.e. `b` relies on `a`); `direction=uses` draws *result to target* ("`a` uses `b`"). The filters above work on the recorded relation, so they are unaffected by the chosen orientation. `\pgph@writeedge` deduplicates.

```
\newcommand\pgph@emitedge[2]{%
  \ifdefstring{\pgph@direction}{uses}%
    {\pgph@writeedge{#1}{#2}}%
    {\pgph@writeedge{#2}{#1}}%
}
\newcommand\pgph@writeedge[2]{%
  \ifcsname pgph@drawn@#1@#2\endcsname\else
    \@namedef{pgph@drawn@#1@#2}{}%
    \immediate\write\pgph@out{\space\space"#1" -> "#2";}%
  \fi
}
```

`\pgph@citeedge` `\pgph@setcitelabel` A `cite:key` target: declare the external node once (keyed, as a stable id, by the BibTeX key), then draw the edge. The node *label* is built by `\pgph@setcitelabel`: it is the key by default, or, with `citelabel=tag`, the printed citation tag (the "[42]"/"[Knu74]" text) recovered from the `\b@`⟨*key*⟩ macro that `\bibcite` writes to the `.aux` (so two runs are needed, and it falls back to the key when no tag is known, e.g. on the first run or with citation packages that do not populate `\b@`⟨*key*⟩). A `\cite` note, when present, is prepended as "⟨*note*⟩, …".

```
\newcommand\pgph@citeedge[2]{%
  \ifcsname pgph@ex@#1\endcsname\else
```

```
    \StrBehind{#2}{cite:}[\pgph@key]%
    \ifcsname pgph@extnode@\pgph@key\endcsname\else
      \@namedef{pgph@extnode@\pgph@key}{}%
      \expandafter\pgph@setcitelabel\expandafter{\pgph@key}%
      \immediate\write\pgph@out{%
        \space\space"c@\pgph@key" [label="\pgph@clabel",\pgph@citestyle];}%
    \fi
    \pgph@emitedge{n#1}{c@\pgph@key}%
  \fi
}
\edef\pgph@obrace{\expandafter\@gobble\string\{}
\newif\ifpgph@natnum
\newcommand\pgph@nil{}
```

`\pgph@citesanitize` neutralizes the wrappers and spacing cruft that citation labels carry, so that an `\edef` of `\b@`⟨*key*⟩ yields plain text: hyperref's `\hyper@@link` (keep its printed last argument) and the boxing/penalty/`\ignorespaces` noise that natbib bakes into author lists.

```
\newcommand\pgph@citesanitize{%
  \let\protect\@empty
  \def\hyper@@link[##1]##2##3##4{##4}%
  \let\unskip\@empty \let\ignorespaces\@empty \let\unhbox\@empty
  \let\penalty\@gobble \let\@M\@empty \let\voidb@x\@empty
  \let\nobreakspace\space \let~\space
  \def\hbox##1{##1}\def\mbox##1{##1}%
}
```

A natbib `\b@`⟨*key*⟩ is `{`⟨*num*⟩`}{`⟨*year*⟩`}{ `⟨*short*⟩`}{`⟨*long*⟩`}`. In numbers mode the tag is the number; in author–year mode it is the short author list and the year.

```
\def\pgph@natpick#1#2#3#4\pgph@nil{%
  \ifpgph@natnum\def\pgph@x{#1}\else\def\pgph@x{#3 #2}\fi}
\newcommand\pgph@natdo{\expandafter\pgph@natpick\pgph@x{}{}{}\pgph@nil}
\newcommand\pgph@setcitelabel[1]{%
  \edef\pgph@cbase{#1}%
  \ifdefstring{\pgph@citelabel}{tag}{%
    \ifcsname b@#1\endcsname
      \global\let\pgph@cbaseg\relax
      \begingroup
        \pgph@citesanitize
        \edef\pgph@x{\csname b@#1\endcsname}%
        \edef\pgph@dx{\detokenize\expandafter{\pgph@x}}%
        \IfBeginWith\pgph@dx\pgph@obrace{%
          \pgph@natnumfalse
          \@ifundefined{ifNAT@numbers}{}{%
            \expandafter\ifx\csname ifNAT@numbers\endcsname\iftrue
              \pgph@natnumtrue\fi}%
          \pgph@natdo
          \edef\pgph@dx{\detokenize\expandafter{\pgph@x}}%
        }{}%
        \ifx\pgph@x\@empty\else
          \IfSubStr\pgph@dx\@backslashchar{}{%
            \global\let\pgph@cbaseg\pgph@x}%
        \fi
      \endgroup
      \ifx\pgph@cbaseg\relax\else\edef\pgph@cbase{[\pgph@cbaseg]}\fi
```

```
    \fi
  }{}%
  \edef\pgph@clabel{%
    \ifcsname pgph@citenote@#1\endcsname
      \csname pgph@citenote@#1\endcsname, \fi
    \pgph@cbase}%
}
```

`\pgph@emit` Open the file, resolve editing rules, write nodes then edges, close, and optionally run Graphviz.

```
\newwrite\pgph@out
\newcommand\pgph@emit{%
  \immediate\openout\pgph@out=\pgph@file.dot\relax
  \immediate\write\pgph@out{digraph proofgraph \pgph@ob}%
  \immediate\write\pgph@out{\space\space rankdir="\pgph@rankdir";}%
  \begingroup
    \pgph@ignores
    \pgph@excludes
    \pgph@nodes
    \pgph@edges
  \endgroup
  \immediate\write\pgph@out{\pgph@cb}%
  \immediate\closeout\pgph@out
  \ifpgph@autorun
    \immediate\write18{\pgph@engine\space -T\pgph@format\space
      \pgph@file.dot -o \pgph@file-proofgraph.\pgph@format}%
  \fi
}
```

The emission is registered at `\begin{document}` rather than at load time so that it is added to the end-document hook *after* packages loaded later (notably apxproof, whose appendix, with the deferred proofs and their references, is built at end of document). This way the graph is written only once that material has been typeset, and the emission does not disturb the verbatim replay of deferred proofs.

```
\AtBeginDocument{\AtEndDocument{\pgph@emit}}
```

`\proofgraph` Include the rendered graph (from a previous run) in `autorun` mode.

```
\newcommand\proofgraph[1][]{%
  \IfFileExists{\pgph@file-proofgraph.\pgph@format}%
    {\includegraphics[#1]{\pgph@file-proofgraph.\pgph@format}}%
    {\PackageWarning{proofgraph}{Rendered graph
       \pgph@file-proofgraph.\pgph@format\space not found; compile with
       shell-escape and the autorun option, then re-run}}%
}
```

⟨/package⟩

Index

Numbers written in italic refer to the page where the corresponding entry is described; numbers underlined refer to the code line of the definition; numbers in roman refer to the code lines where the entry is used.

Symbols

S

T

U

V

Change History

v0.1.0